

TECSUP · Carrera de Big Data y Ciencia de Datos

FUNDAMENTOS DE GESTIÓN DE DATOS

UNIDAD 2

PYTHON PARA DATOS, LIMPIEZA Y MACHINE LEARNING BÁSICO

Material de estudio · Semanas 5 a 8

De la base de datos a la decisión basada en modelos

Semana	Tema
5	Python y Pandas para Datos
6	Limpieza de Datos
7	ML Supervisado: Regresión y Clasificación
8	ML No Supervisado: Clustering (K-Means)

Docente: Pilar Rocío Sayán Mejía

Periodo académico: 2026-II

Contenido

Introducción a la Unidad 2.....	3
¿Qué aprenderemos?.....	3
Caso transversal: RutaMarket, un año después.....	3
Semana 5 · Python y Pandas para Datos.....	4
5.1 Google Colab: el entorno de trabajo.....	4
5.2 Lo esencial de Python para datos.....	4
5.3 pandas: DataFrame y Series.....	4
5.4 Leer datos desde SQLite y CSV.....	5
5.5 Explorar los datos: el análisis exploratorio (EDA).....	6
5.6 Visualizar: matplotlib y seaborn.....	7
Semana 6 · Limpieza de Datos.....	9
6.1 ¿Por qué limpiar?.....	9
6.2 Diagnóstico inicial.....	9
6.3 Tipos incorrectos y valores imposibles.....	9
6.4 Inconsistencias de texto.....	10
6.5 Registros duplicados.....	10
6.6 Datos faltantes: detección e imputación.....	10
6.7 Valores atípicos: boxplot, z-score e IQR.....	11
6.8 Normalización: Min-Max y estandarización.....	13
6.9 Guardar el dataset limpio en SQLite.....	14
Semana 7 · ML Supervisado: Regresión y Clasificación.....	15
7.1 ¿Qué es Machine Learning?.....	15
7.2 El proceso CRISP-DM.....	15
7.3 Datos de entrenamiento y de prueba.....	15
7.4 Regresión lineal simple.....	16
7.5 Regresión lineal múltiple.....	17
7.6 Overfitting y underfitting.....	17
7.7 Clasificación: concepto e interpretación.....	18
7.8 Uso responsable de los modelos.....	19
Semana 8 · ML No Supervisado: Clustering con K-Means.....	20
8.1 Aprendizaje no supervisado: ¿cuándo usar clustering?.....	20
8.2 Cómo funciona K-Means.....	20
8.3 Preparar los datos y elegir k: el método del codo.....	20
8.4 Interpretar los clusters.....	21
8.5 Guardar los segmentos en la base de datos.....	22
8.6 Limitaciones de K-Means.....	22
Integración de la Unidad 2.....	23
Mapa del recorrido.....	23
Caso integrador.....	23
Relación con el proyecto PMD1.....	23
Checklist de dominio de la Unidad 2.....	24
Conclusiones.....	24
Referencias bibliográficas.....	24

Introducción a la Unidad 2

En la Unidad 1 aprendimos a comprender, organizar y consultar datos: vimos por qué la calidad importa, escribimos consultas SQL y diseñamos un modelo relacional. Esa base responde preguntas como *¿cuánto vendimos?* o *¿quiénes compraron?* La Unidad 2 da el siguiente paso: **preparar y analizar** los datos para responder preguntas más exigentes, como *¿qué problemas tienen nuestros datos?*, *¿qué variables se relacionan con el gasto de un cliente?* o *¿qué grupos de clientes existen?*

Para ello incorporamos **Python** y su librería **pandas**, que trabajan junto a SQL: SQL extrae los datos de la base, pandas los explora y limpia, y **scikit-learn** construye modelos de *Machine Learning* básicos. El resultado vuelve a guardarse en la base de datos, cerrando el ciclo de vida del dato estudiado en la Semana 1.



¿Qué aprenderemos?

Semana	Pregunta que responde	Al terminar, podrás...
5. Python y Pandas	¿Cómo llevo los datos de la base a un entorno de análisis y los exploro?	Leer datos desde SQLite y CSV, explorarlos con pandas e interpretar gráficos de distribución, correlación y nulos.
6. Limpieza de Datos	¿Qué problemas tienen los datos y cómo los corrijo sin distorsionarlos?	Detectar y tratar nulos, duplicados, tipos incorrectos y atípicos; normalizar variables y guardar el resultado en SQLite.
7. ML Supervisado	¿Puedo estimar un valor o una categoría a partir de otras variables?	Entrenar y evaluar una regresión lineal, interpretar sus métricas y reconocer el sobreajuste y la lógica de la clasificación.
8. ML No Supervisado	¿Qué grupos naturales existen en mis datos?	Aplicar K-Means, elegir k con el método del codo e interpretar los segmentos para decisiones de negocio.

Cómo leer este material. Como en la Unidad 1, los conceptos combinan concepto, contexto, ejemplo, aplicación, interpretación y conexión, y el código sigue el orden *pregunta de negocio* → *código* → *resultado* → *interpretación*. No se trata de memorizar sintaxis, sino de entender qué hace cada paso y qué significa su resultado.

Caso transversal: RutaMarket, un año después

Continuamos con **RutaMarket S.A.C.**, la tienda en línea *ficticia* de la Unidad 1. Un año después, la empresa ha crecido y su base de datos `rutamarket.db` registra la operación de **2025**: clientes, productos, categorías, pedidos y detalle de pedidos, con el mismo modelo relacional diseñado en la Semana 4. Como parte de la tabla de clientes proviene de una migración desde hojas de cálculo, los datos traen problemas de calidad que deberemos descubrir y corregir. Los datos son **sintéticos**, generados con fines académicos. Todo el código fue ejecutado sobre `rutamarket.db` (pandas 2.x y scikit-learn): los resultados mostrados son los que obtendrás al reproducirlo. Los montos están en soles (S/).

La base contiene **408 clientes** (con ciudad, edad, canal y fecha de registro, correo y visitas mensuales a la web), **3 004 pedidos** (fecha, método de pago y estado: Entregado o Devuelto), **4 737 líneas de detalle**, **20 productos** y **4 categorías**.

Semana	Cómo interviene el caso
5	Leemos las tablas desde SQLite con pandas y hacemos un primer análisis exploratorio.
6	Corregimos los problemas de calidad y construimos una tabla limpia de clientes (<code>clientes_modelo</code>) en la misma base.
7	Estimamos el gasto anual de un cliente a partir de su actividad en la web y discutimos la clasificación de clientes de alto valor.
8	Segmentamos a los clientes con K-Means y guardamos el segmento de cada uno en la base de datos.

Semana 5 · Python y Pandas para Datos

Pregunta guía: ¿cómo llevo los datos de una base de datos a un entorno de análisis y obtengo una primera visión de ellos?

SQL es excelente para extraer y resumir datos, pero el análisis suele requerir pasos adicionales: revisar la calidad de cada columna, transformar valores, graficar y, más adelante, construir modelos. Python, con pandas, permite hacer todo eso en un mismo lugar y dejar registrado cada paso, de modo que el análisis pueda repetirse y revisarse.

5.1 Google Colab: el entorno de trabajo

Concepto. **Google Colab** es un entorno gratuito que ejecuta código Python en servidores de Google desde el navegador, sin instalar nada en la computadora. Trabaja con **notebooks** (archivos `.ipynb`), documentos que combinan celdas de código, sus resultados y celdas de texto.

Tipo de celda	Para qué sirve	Ejemplo
Código	Escribir y ejecutar Python (Shift + Enter)	<code>clientes.head()</code>
Texto (Markdown)	Explicar el objetivo, las decisiones y las conclusiones	<code>## Hallazgos del EDA</code> crea un subtítulo; <code>**texto**</code> , negrita

Aplicación. Un notebook bien hecho se lee como un informe: cada bloque de código va precedido de una celda de texto que explica qué pregunta responde y seguido de su interpretación. Las celdas se ejecutan en orden y comparten variables; si se reinicia el entorno, hay que volver a ejecutarlas desde el inicio. Para trabajar con el caso, sube `rutamarket.db` al panel *Archivos* de Colab (los archivos subidos se borran al cerrar la sesión) o léelo desde Google Drive.

5.2 Lo esencial de Python para datos

Para esta unidad basta con un conjunto pequeño de elementos de Python:

```
ciudad = 'Lima'           # str: texto
visitas = 12              # int: número entero
ticket = 254.80          # float: número decimal
activo = True             # bool: verdadero o falso
ciudades = ['Lima', 'Cusco', 'Piura'] # lista: colección ordenada
cliente = {'id': 1, 'ciudad': 'Lima'}  # diccionario: pares clave-valor
print(len(ciudades), cliente['ciudad']) # funciones y acceso por clave
```

```
3 Lima
```

También usaremos **librerías**: código ya escrito que se incorpora con `import`. Las principales son **pandas** (tablas), **matplotlib** y **seaborn** (gráficos) y **scikit-learn** (modelos). En Colab ya vienen instaladas; por convención se importan con alias: `import pandas as pd`. Un **método** es una función asociada a un objeto y se invoca con un punto: `clientes.head()`.

5.3 pandas: DataFrame y Series

Concepto. pandas organiza los datos en dos estructuras. Un **DataFrame** es una tabla en memoria, con filas y columnas con nombre, equivalente a la tabla de una base de datos o al resultado de una consulta SQL. Una **Series** es una sola columna de un DataFrame.

Conexión. Muchas operaciones de pandas tienen un equivalente directo en lo aprendido en la Unidad 1:

Operación	SQL (Unidad 1)	pandas (Unidad 2)
Ver algunas filas	<code>SELECT * FROM clientes LIMIT 5;</code>	<code>clientes.head()</code>
Filtrar filas	<code>WHERE ciudad = 'Lima'</code>	<code>clientes[clientes['ciudad'] == 'Lima']</code>
Valores distintos	<code>SELECT DISTINCT ciudad</code>	<code>clientes['ciudad'].unique()</code>
Contar por grupo	<code>GROUP BY ciudad + COUNT(*)</code>	<code>clientes['ciudad'].value_counts()</code>
Agregar por grupo	<code>GROUP BY id_cliente + SUM(monto)</code>	<code>pedidos.groupby('id_cliente')['monto'].sum()</code>
Unir tablas	<code>JOIN ... ON ...</code>	<code>pedidos.merge(clientes, on='id_cliente')</code>

Interpretación. SQL y pandas no compiten: SQL trabaja dentro de la base de datos y es ideal para extraer y combinar; pandas trabaja con los datos ya cargados en memoria y es ideal para explorar, transformar y preparar datos para modelos.

5.4 Leer datos desde SQLite y CSV

La librería `sqlite3` abre la conexión con la base de datos y `pd.read_sql_query()` ejecuta una consulta SQL y devuelve el resultado como DataFrame.

PREGUNTA DE NEGOCIO

¿Cuántos clientes y columnas tiene la tabla de clientes, y qué aspecto tienen sus registros?

CÓDIGO PYTHON

```
import sqlite3
import pandas as pd

con = sqlite3.connect('rutamarket.db')
clientes = pd.read_sql_query('SELECT * FROM clientes', con)
print(clientes.shape)

clientes[['id_cliente', 'ciudad', 'edad', 'canal_registro', 'visitas_web_mes']].head()
```

RESULTADO

(408, 8)

id_cliente	ciudad	edad	canal_registro	visitas_web_mes
1	Lima	33	Web	17
2	Lima	33	App	10
3	Lima	41	Web	9
4	Lima	34	App	4
5	Lima	51	Web	4

INTERPRETACIÓN

`shape` indica 408 filas y 8 columnas. La vista preliminar muestra, por ejemplo, que el cliente 1 es de Lima, se registró por la web y visita la tienda 17 veces al mes. Es la misma tabla `clientes` de la Unidad 1, ahora cargada como DataFrame.

PREGUNTA DE NEGOCIO

¿Cuál es el monto total de cada pedido?

CÓDIGO PYTHON

```
consulta = '''
SELECT p.id_pedido, p.id_cliente, p.fecha_pedido, p.metodo_pago, p.estado,
       SUM(d.cantidad * d.precio_unitario) AS monto
FROM pedidos AS p
JOIN detalle_pedido AS d ON p.id_pedido = d.id_pedido
GROUP BY p.id_pedido;
'''

pedidos = pd.read_sql_query(consulta, con)
print(pedidos.shape)
```

RESULTADO

(3004, 6)

INTERPRETACIÓN

La consulta SQL, idéntica a las de la Semana 3, une `pedidos` con `detalle_pedido` y calcula el monto de cada pedido. El resultado llega a Python como un DataFrame de 3 004 pedidos y 6 columnas. Es la forma habitual de trabajar: **SQL prepara la tabla de análisis y pandas continúa desde ahí.**

CÓDIGO PYTHON

```
productos = pd.read_csv('productos.csv')
productos.head(3)
```

RESULTADO

id_producto	nombre_producto	id_categoria	precio_unitario
1	Audífonos inalámbricos	1	120.00
2	Mouse inalámbrico	1	45.00
3	Teclado mecánico	1	210.00

INTERPRETACIÓN

Los CSV se leen con `pd.read_csv()`; conviene revisar el separador (`sep`), la codificación (`encoding`) y los tipos de cada columna.

5.5 Explorar los datos: el análisis exploratorio (EDA)

Concepto. El **análisis exploratorio de datos** (EDA, por *Exploratory Data Analysis*) consiste en examinar un conjunto de datos para entender su estructura, detectar problemas de calidad y descubrir patrones antes de sacar conclusiones o construir modelos (Tukey, 1977). Responde preguntas como: ¿qué tipo tiene cada columna?, ¿hay vacíos?, ¿qué valores son frecuentes?, ¿hay valores extraños?

PREGUNTA DE NEGOCIO

¿Qué tipo de dato tiene cada columna y cuántos valores no nulos hay?

CÓDIGO PYTHON

```
clientes.info()
```

RESULTADO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 408 entries, 0 to 407
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   id_cliente          408 non-null   int64
 1   nombre              408 non-null   object
 2   ciudad              396 non-null   object
 3   edad                408 non-null   object
 4   canal_registro      408 non-null   object
 5   fecha_registro      408 non-null   object
 6   email               408 non-null   object
 7   visitas_web_mes     408 non-null   int64
dtypes: int64(2), object(6)
memory usage: 25.6+ KB
```

INTERPRETACIÓN

`ciudad` tiene 396 valores no nulos de 408: faltan 12. Más importante aún, `edad` aparece como `object` (texto) en lugar de número, y lo mismo ocurre con `fecha_registro`. Esto indica que algunos valores de esas columnas no son números o fechas válidas: un problema que corregiremos en la Semana 6.

Nota: en pandas 3.x el texto se muestra con el tipo `str` en lugar de `object`; el significado es el mismo.

PREGUNTA DE NEGOCIO

¿Cómo se distribuye el monto de los pedidos?

CÓDIGO PYTHON

```
pedidos['monto'].describe().round(2)
```

RESULTADO

estadístico	monto
count	3,004.00
mean	312.57
std	555.77
min	35.00
25%	99.00
50%	199.00
75%	339.00
max	19,070.00

INTERPRETACIÓN

El pedido mediano es de S/ 199.00 (50%) y la mitad central de los pedidos está entre S/ 99 y S/ 339. Sin embargo, el máximo es S/ 19 070 y el promedio (S/ 312.57) está muy por encima de la mediana. Un máximo tan alejado puede ser una compra real muy grande o un error de registro; el EDA no lo decide, pero **lo señala** para investigarlo.

PREGUNTA DE NEGOCIO

¿Cómo están registradas las ciudades?

CÓDIGO PYTHON

```
clientes['ciudad'].value_counts(dropna=False)
```

RESULTADO

ciudad	count	ciudad	count
Lima	153	LIMA	2
Arequipa	63	arequipa	1
Cusco	45	AREQUIPA	1
Trujillo	42	CHICLAYO	1
Chiclayo	38	trujillo	1
Piura	37	TRUJILLO	1
NaN	12	chiclayo	1
PIURA	3	CUSCO	1
lima	3	cusco	1
piura	2		

INTERPRETACIÓN

`value_counts()` revela problemas de **consistencia** (Semana 1): "Lima", "lima" y " LIMA " se cuentan como ciudades distintas, y hay 12 clientes sin ciudad (NaN). Si calculáramos ventas por ciudad con estos datos, Lima quedaría subestimada. (Las comillas marcan los valores con espacios sobrantes.)

5.6 Visualizar: matplotlib y seaborn

Un gráfico muestra de un vistazo lo que una tabla esconde. **matplotlib** es la librería base de gráficos en Python y **seaborn** se apoya en ella para producir gráficos estadísticos con menos código. Primero resumimos los pedidos por cliente para estudiar la relación entre visitas, número de pedidos y gasto:

CÓDIGO PYTHON

```
resumen = (pedidos.groupby('id_cliente')
            .agg(n_pedidos='id_pedido', 'count'), gasto=('monto', 'sum'))
            .reset_index()
            .merge(clientes[['id_cliente', 'visitas_web_mes']],
                  on='id_cliente')
resumen[['visitas_web_mes', 'n_pedidos', 'gasto']].corr().round(2)
```

RESULTADO

index	visitas_web_mes	n_pedidos	gasto
visitas_web_mes	1.00	0.76	0.55
n_pedidos	0.76	1.00	0.66
gasto	0.55	0.66	1.00

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, axs = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(pedidos['monto'], bins=40, ax=axs[0]) # distribución
sns.heatmap(resumen[['visitas_web_mes', 'n_pedidos', 'gasto']].corr(), # correlación
            annot=True, cmap='Blues', ax=axs[1])
nulos = pd.concat([clientes.isnull().sum(), pedidos.isnull().sum()])
nulos.plot.barh(ax=axs[2]) # nulos
plt.tight_layout()
```

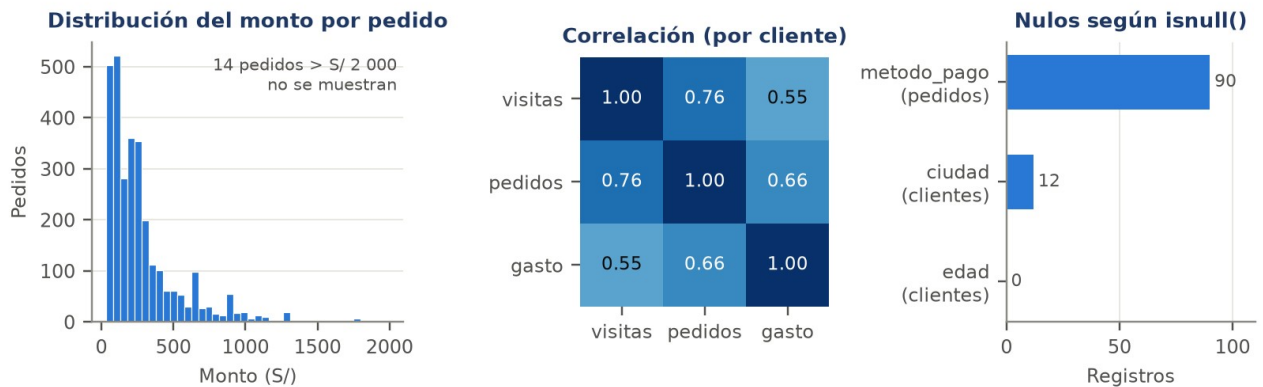


Figura 1. Tres gráficos básicos del EDA de RutaMarket: distribución, correlación y valores nulos.

Gráfico	Qué muestra	Interpretación para RutaMarket
Distribución (histograma)	Cuántos pedidos hay en cada rango de monto	El 84 % de los pedidos es menor a S/ 500 y la distribución tiene una cola larga a la derecha: pocos pedidos muy grandes elevan el promedio.
Correlación (mapa de calor)	Fuerza de la relación lineal entre pares de variables, de -1 a 1	Los clientes que más visitan la web hacen más pedidos (0.76) y gastan más (0.55). Correlación no implica causalidad, pero orienta el modelo de la Semana 7.
Nulos (barras)	Cuántos valores faltan en cada columna	Faltan 90 métodos de pago y 12 ciudades. <code>edad</code> muestra 0 nulos, aunque <code>info()</code> indicó que es texto: los vacíos escritos como texto no se detectan como nulos.

Cierre de la Semana 5

Lo que debes dominar

- Trabajar en un notebook de Colab combinando celdas de código y de texto que documentan el análisis.
- Usar los elementos básicos de Python (tipos, listas, diccionarios, funciones, librerías) necesarios para analizar datos.
- Leer datos desde SQLite con `pd.read_sql_query()` y desde CSV con `pd.read_csv()`.
- Explorar un DataFrame con `head()`, `info()`, `describe()` y `value_counts()` e identificar problemas de calidad.
- Construir e interpretar gráficos de distribución, correlación y valores nulos.

Comprueba tu aprendizaje

- ¿Qué ventaja tiene extraer datos con una consulta SQL dentro de `pd.read_sql_query()` en lugar de leer todas las tablas por separado?
- `describe()` muestra una media de S/ 312.57 y una mediana de S/ 199.00. ¿Qué sugiere esa diferencia sobre la distribución?
- Una columna numérica aparece como `object` en `info()`. ¿Qué podría estar ocurriendo y por qué es un problema?
- La correlación entre visitas y gasto es 0.55. ¿Permite afirmar que aumentar las visitas aumentará el gasto? Justifica.

Conexión con la Semana 6

Semana 5 → ya sabemos cargar los datos en pandas y explorarlos; el EDA detectó nulos, textos inconsistentes, tipos incorrectos y montos sospechosos.

Semana 6 → ahora **corregiremos** esos problemas de forma justificada y guardaremos un conjunto de datos limpio en la base de datos, listo para construir modelos.

Semana 6 · Limpieza de Datos

Pregunta guía: ¿cómo corrijo los problemas de los datos sin distorsionar la realidad que representan?

6.1 ¿Por qué limpiar?

En la Semana 1 vimos el principio *Garbage In, Garbage Out*. La limpieza es la etapa del ciclo de vida del dato en la que se detectan y tratan los problemas de calidad antes del análisis (Ilyas y Chu, 2019). No consiste en "borrar lo que molesta": cada decisión cambia los resultados y debe justificarse y documentarse.

Problema en RutaMarket	Dimensión de calidad (Unidad 1)	Técnica en pandas
Edades vacías, "N.D." o 150	Complejidad, validez	<code>pd.to_numeric(errors='coerce')</code> , reglas de rango, imputación
"Lima", "lima", " LIMA "	Consistencia	<code>.str.strip().str.title()</code>
Cientes y pedidos registrados dos veces	Unicidad	<code>duplicated()</code> , <code>drop_duplicates()</code>
Métodos de pago faltantes	Complejidad	<code>isnull()</code> , <code>fillna()</code>
Precios cien veces mayores que el catálogo	Exactitud	Detección de atípicos y comparación con tabla de referencia

6.2 Diagnóstico inicial

PREGUNTA DE NEGOCIO

¿Cuántos valores faltan en cada columna?

CÓDIGO PYTHON

```
print(clientes.isnull().sum())
print('Pedidos sin método de pago:', pedidos['metodo_pago'].isnull().sum())
```

RESULTADO

```
id_cliente      0
nombre          0
ciudad         12
edad            0
canal_registro  0
fecha_registro  0
email           0
visitas_web_mes 0
dtype: int64
Pedidos sin método de pago: 90
```

INTERPRETACIÓN

`isnull()` detecta 12 ciudades y 90 métodos de pago faltantes, pero **ninguna edad**. Como vimos en el EDA, las edades vacías están guardadas como texto (" " o "N.D.") y pandas no las reconoce como nulos. Un diagnóstico basado solo en `isnull()` habría pasado por alto el problema.

6.3 Tipos incorrectos y valores imposibles

PREGUNTA DE NEGOCIO

¿Cuántas edades no son números válidos?

CÓDIGO PYTHON

```
clientes['edad'] = pd.to_numeric(clientes['edad'], errors='coerce')
clientes['fecha_registro'] = pd.to_datetime(clientes['fecha_registro'])
print(clientes['edad'].dtype, clientes['edad'].isnull().sum())

print(clientes['edad'].describe().round(1)[['min', 'max']])
clientes.loc[(clientes['edad'] < 18) | (clientes['edad'] > 100), 'edad'] = None
```

RESULTADO

```
float64 34

min      18.0
max     150.0
Name: edad, dtype: float64
```

INTERPRETACIÓN

`pd.to_numeric(..., errors='coerce')` convierte a número lo que puede y transforma en `NaN` lo que no: aparecen 34 edades faltantes que antes estaban ocultas. `pd.to_datetime()` convierte `fecha_registro` en fecha, lo que permite calcular antigüedades.

Además, el máximo de 150 años es imposible. Una **regla de validez** (edad entre 18 y 100) lo convierte en faltante: es preferible reconocer que no se conoce el dato a mantener un valor que se sabe falso.

6.4 Inconsistencias de texto

PREGUNTA DE NEGOCIO

¿Cuántos clientes hay realmente en cada ciudad?

CÓDIGO PYTHON

```
clientes['ciudad'] = clientes['ciudad'].str.strip().str.title()
clientes['ciudad'].value_counts(dropna=False)
```

RESULTADO

ciudad	count
Lima	158
Arequipa	65
Cusco	47
Trujillo	44
Piura	42
Chiclayo	40
None	12

INTERPRETACIÓN

`strip()` elimina espacios sobrantes y `title()` unifica el formato ("LIMA" y "lima" pasan a "Lima"). Lima tenía 153 clientes con la escritura correcta; tras la corrección son 158. Las variantes desaparecen y los conteos por ciudad ya son confiables.

6.5 Registros duplicados

PREGUNTA DE NEGOCIO

¿Hay clientes o pedidos registrados más de una vez?

CÓDIGO PYTHON

```
print('Clientes con email repetido:', clientes.duplicated(subset='email').sum())
clientes = clientes.drop_duplicates(subset='email', keep='first')
print(clientes.shape)

clave = ['id_cliente', 'fecha_pedido', 'monto', 'estado']
print('Pedidos duplicados:', pedidos.duplicated(subset=clave).sum())
pedidos = pedidos.drop_duplicates(subset=clave, keep='first')
print(pedidos.shape)
```

RESULTADO

```
Clientes con email repetido: 8
(400, 8)
```

```
Pedidos duplicados: 15
(2989, 6)
```

INTERPRETACIÓN

Ocho clientes aparecen dos veces con distinto `id_cliente` pero el mismo correo: la migración les asignó un código nuevo. Por eso los duplicados se buscan con un criterio de negocio (`subset`), no por el identificador.

En pedidos, 15 registros coinciden en cliente, fecha, monto y estado: corresponden a pedidos registrados dos veces. Antes de eliminar, conviene confirmar con el área responsable, porque un cliente podría hacer legítimamente dos compras iguales el mismo día.

6.6 Datos faltantes: detección e imputación

Concepto. Imputar es reemplazar un valor faltante por una estimación razonable. La estrategia depende del tipo de variable y de su distribución:

Estrategia	Cuándo usarla	Riesgo
Eliminar filas	Pocos faltantes y sin un patrón	Perder información y sesgar la muestra si los faltantes no son al azar
Media	Variable numérica con distribución simétrica	Se desplaza con valores extremos
Mediana	Variable numérica asimétrica o con atípicos	Reduce la variabilidad si hay muchos faltantes
Moda	Variable categórica	Sobrerrepresenta la categoría más frecuente
Categoría explícita ("No registrada")	Categórica cuando el faltante tiene significado propio	Agrega una categoría que no es un valor real

PREGUNTA DE NEGOCIO

¿Qué valores usamos para completar los datos faltantes?

CÓDIGO PYTHON

```
print('Media:', round(clientes['edad'].mean(), 1),
      '| Mediana:', clientes['edad'].median())
clientes['edad'] = clientes['edad'].fillna(clientes['edad'].median())
clientes['ciudad'] = clientes['ciudad'].fillna('No registrada')
moda_pago = pedidos['metodo_pago'].mode()[0]
pedidos['metodo_pago'] = pedidos['metodo_pago'].fillna(moda_pago)
print('Moda método de pago:', moda_pago)
```

RESULTADO

```
Media: 35.6 | Mediana: 35.0
Moda método de pago: Tarjeta
```

INTERPRETACIÓN

La media (35.6) y la mediana (35.0) de la edad son casi iguales porque su distribución es simétrica; se elige la mediana por ser más robusta. Para **ciudad** se usa la categoría "No registrada" en lugar de la moda: asignar "Lima" a 12 clientes inventaría ventas en Lima. Para el método de pago se usa la moda ("Tarjeta"), porque en esta etapa solo se necesita una variable completa.

Cada imputación es un supuesto; debe registrarse para que quien use los datos sepa qué valores son estimados.

6.7 Valores atípicos: boxplot, z-score e IQR

Concepto. Un **valor atípico** (*outlier*) se aleja notablemente del resto. Puede ser un error o un caso real poco frecuente; detectarlo no equivale a eliminarlo. Hay tres herramientas habituales:

Método	Regla	Característica
Boxplot	La caja va de Q1 a Q3; los puntos fuera de los "bigotes" son atípicos	Visual; usa la misma regla que el IQR
Rango intercuartílico (IQR)	$IQR = Q3 - Q1$; atípico si $< Q1 - 1.5 \cdot IQR$ o $> Q3 + 1.5 \cdot IQR$ (Tukey, 1977)	Robusto: no depende de la media
z-score	$z = (x - media) / desviación\ estándar$; atípico si $ z > 3$	Supone una distribución aproximadamente simétrica

PREGUNTA DE NEGOCIO

¿Qué pedidos tienen montos atípicos?

CÓDIGO PYTHON

```
q1, q3 = pedidos['monto'].quantile([0.25, 0.75])
iqr = q3 - q1
lim_inf, lim_sup = q1 - 1.5 * iqr, q3 + 1.5 * iqr
atipicos_iqr = pedidos[(pedidos['monto'] < lim_inf) | (pedidos['monto'] > lim_sup)]
z = (pedidos['monto'] - pedidos['monto'].mean()) / pedidos['monto'].std()
print('Límite superior IQR:', round(lim_sup, 2))
print('Atípicos por IQR:', len(atipicos_iqr),
      '| por z-score (|z|>3):', (z.abs() > 3).sum())

pedidos.nlargest(8, 'monto')[['id_pedido', 'id_cliente', 'monto']]
```

RESULTADO

```
Límite superior IQR: 699.0
Atípicos por IQR: 257 | por z-score (|z|>3): 15
```

id_pedido	id_cliente	monto
12735	362	19,070.00
10121	17	12,059.00
12796	374	9,347.00
11876	248	7,995.00
11000	127	3,689.80
11945	257	3,500.00
10062	8	3,178.00
10643	86	2,697.00

INTERPRETACIÓN

Los métodos discrepan: el IQR marca 257 pedidos (todo lo que supera S/ 699) y el z-score solo 15. La distribución de montos es asimétrica: muchos pedidos pequeños y algunos grandes. Por eso el IQR señala como atípicas muchas compras normales de productos caros, mientras que el z-score, inflado por los valores extremos, solo detecta los más alejados.

Los primeros montos de la lista (S/ 19 070, S/ 12 059...) merecen revisión. Para saber si son errores hay que compararlos con una fuente de referencia.

PREGUNTA DE NEGOCIO

¿Los montos extremos se deben a precios mal registrados?

CÓDIGO PYTHON

```
detalle = pd.read_sql_query('SELECT * FROM detalle_pedido', con)
catalogo = pd.read_sql_query('SELECT id_producto, precio_unitario AS precio_catalogo
                             FROM productos', con)
detalle = detalle.merge(catalogo, on='id_producto')
errores = detalle[detalle['precio_unitario'] != detalle['precio_catalogo']]
print('Líneas con precio distinto al catálogo:', len(errores))
errores[['id_pedido', 'id_producto', 'precio_unitario', 'precio_catalogo']]

detalle['precio_unitario'] = detalle['precio_catalogo']
montos = (detalle.assign(subtotal=detalle['cantidad'] * detalle['precio_unitario'])
           .groupby('id_pedido', as_index=False)['subtotal'].sum())
pedidos = (pedidos.drop(columns='monto')
           .merge(montos, on='id_pedido')
           .rename(columns={'subtotal': 'monto'}))
print(pedidos['monto'].describe().round(2)[['mean', '50%', 'max']])
```

RESULTADO

Líneas con precio distinto al catálogo: 6

id_pedido	id_producto	precio_unitario	precio_catalogo
10121	1	12,000.00	120.00
11000	19	3,500.00	35.00
11876	18	7,900.00	79.00
11945	19	3,500.00	35.00
12735	6	18,990.00	189.90
12796	2	4,500.00	45.00

```
mean      294.84
50%       199.00
max       3178.00
Name: monto, dtype: float64
```

INTERPRETACIÓN

Al comparar cada línea con el catálogo aparecen exactamente 6 precios cien veces mayores que el de lista (por ejemplo, 18 990 en lugar de 189.90): son errores de digitación en los que se omitió el punto decimal. Se corrigen con el precio del catálogo y se recalcula el monto de cada pedido.

Tras la corrección, el máximo baja a S/ 3 178, que corresponde a compras reales de productos caros (por ejemplo, bicicletas estáticas). Esos pedidos se **conservan**: eliminarlos borraría a los mejores clientes. La Figura 2 muestra el antes y el después.

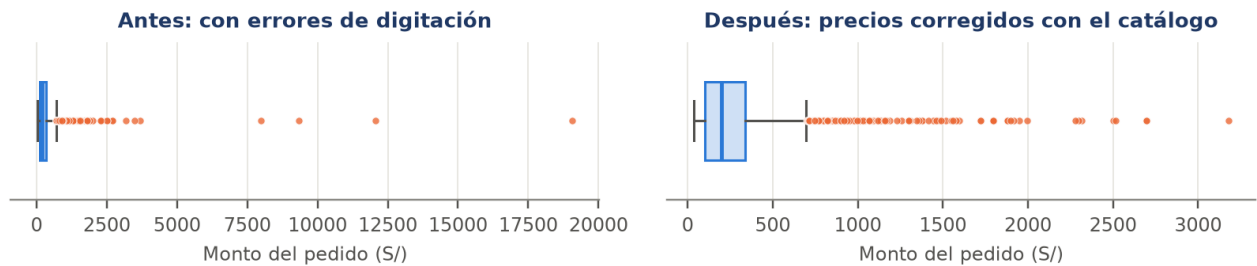


Figura 2. Boxplot del monto por pedido antes y después de corregir los errores de precio.

6.8 Normalización: Min-Max y estandarización

Concepto. Las variables suelen estar en escalas distintas: el número de pedidos va de 1 a 23 y el ticket promedio de decenas a miles de soles. Los métodos basados en distancias, como K-Means (Semana 8), darían más peso a la variable de mayor escala. **Normalizar** las lleva a una escala común.

	Min-Max	Estandarización (z-score)
Fórmula	$x' = (x - \text{mín}) / (\text{máx} - \text{mín})$	$x' = (x - \text{media}) / \text{desviación estándar}$
Resultado	Valores entre 0 y 1	Media 0 y desviación estándar 1
Sensibilidad a atípicos	Alta: un extremo comprime al resto	También se ve afectada, en menor grado
En scikit-learn	<code>MinMaxScaler()</code>	<code>StandardScaler()</code>

Para aplicarla necesitamos una tabla con **una fila por cliente**. Se construye igual que un `GROUP BY` de la Unidad 1: contando pedidos, sumando montos y calculando cuántos días pasaron desde la última compra. Solo se consideran los pedidos entregados.

CÓDIGO PYTHON

```
pedidos['fecha_pedido'] = pd.to_datetime(pedidos['fecha_pedido'])
validos = pedidos[pedidos['estado'] == 'Entregado']
corte = pd.Timestamp('2025-12-31')
modelo = (validos.groupby('id_cliente')
          .agg(pedidos_2025=('id_pedido', 'count'),
              gasto_2025=('monto', 'sum'),
              ultima_compra=('fecha_pedido', 'max'))
          .reset_index())
modelo['ticket_promedio'] = (modelo['gasto_2025'] / modelo['pedidos_2025']).round(2)
modelo['dias_sin_comprar'] = (corte - modelo['ultima_compra']).dt.days
atributos = ['id_cliente', 'ciudad', 'edad', 'visitas_web_mes', 'fecha_registro']
modelo = modelo.merge(clientes[atributos], on='id_cliente')
dias = (corte - modelo['fecha_registro']).dt.days
modelo['antigüedad_meses'] = (dias / 30.4).round(1)
modelo = modelo.drop(columns=['ultima_compra', 'fecha_registro'])
print(modelo.shape)

from sklearn.preprocessing import MinMaxScaler, StandardScaler
cols = ['pedidos_2025', 'ticket_promedio']
minmax = MinMaxScaler().fit_transform(modelo[cols])
estandar = StandardScaler().fit_transform(modelo[cols])
print('Min-Max -> mín:', minmax.min(axis=0).round(2),
      'máx:', minmax.max(axis=0).round(2))
print('Estandar -> media:', estandar.mean(axis=0).round(2),
      'desv.:', estandar.std(axis=0).round(2))
```

RESULTADO

```
(398, 9)
```

```
Min-Max -> mín: [0. 0.] máx: [1. 1.]
Estandar -> media: [ 0. -0.] desv.: [1. 1.]
```

INTERPRETACIÓN

La tabla `modelo` tiene 398 clientes: dos clientes solo tienen pedidos devueltos y no generan gasto. Tras escalar, Min-Max deja ambas variables entre 0 y 1, y la estandarización, con media 0 y desviación 1. Ninguna variable domina ya por su escala.

6.9 Guardar el dataset limpio en SQLite

PREGUNTA DE NEGOCIO

¿Cómo dejamos los datos limpios disponibles para el resto del equipo?

CÓDIGO PYTHON

```
clientes.to_sql('clientes_limpio', con, if_exists='replace', index=False)
pedidos.to_sql('pedidos_limpio', con, if_exists='replace', index=False)
modelo.to_sql('clientes_modelo', con, if_exists='replace', index=False)
pd.read_sql_query('SELECT COUNT(*) AS filas, ROUND(AVG(gasto_2025), 2) AS gasto_medio
FROM clientes_modelo', con)
```

RESULTADO

filas	gasto_medio
398	2,112.60

INTERPRETACIÓN

`to_sql()` guarda cada DataFrame como una tabla nueva en la misma base de datos, sin modificar las tablas originales. Así se conserva la trazabilidad: los datos originales y los limpios pueden compararse. La verificación con SQL confirma 398 clientes con un gasto medio de S/ 2 112.60.

Paso	Antes	Después
Clientes	408 (8 duplicados)	400
Pedidos	3 004 (15 duplicados)	2 989
Edades faltantes	0 detectadas (36 ocultas o imposibles)	0 (imputadas con la mediana)
Ciudades	18 formas de escribir 6 ciudades y 12 vacías	6 ciudades + "No registrada"
Monto máximo	S/ 19 070 (error de digitación)	S/ 3 178 (compra real)

Cierre de la Semana 6

Lo que debes dominar

- Diagnosticar nullos, incluidos los "ocultos" como texto, y corregir tipos incorrectos con `to_numeric()` y `to_datetime()`.
- Unificar textos inconsistentes y eliminar duplicados con un criterio de negocio.
- Elegir entre eliminar, imputar con media, mediana o moda, o usar una categoría explícita, justificando la decisión.
- Detectar atípicos con boxplot, IQR y z-score, y distinguir un error de un caso real antes de actuar.
- Normalizar variables con Min-Max o estandarización y guardar el resultado en SQLite con `to_sql()`.

Comprueba tu aprendizaje

1. ¿Por qué `isnull()` no detectó las edades vacías? ¿Qué paso previo lo permitió?
2. Una variable de ingresos tiene media S/ 4 800 y mediana S/ 2 900. ¿Imputarías con la media o con la mediana? ¿Por qué?
3. El IQR marcó 257 pedidos atípicos. ¿Deberían eliminarse? Explica qué información usarías para decidir.
4. ¿Por qué conviene guardar el dataset limpio como una tabla nueva en lugar de sobrescribir la tabla original?

Conexión con la Semana 7

Semana 6 → ya tenemos una tabla limpia, `clientes_modelo`, con una fila por cliente, guardada en la base de datos.

Semana 7 → con datos confiables podemos construir el primer **modelo de Machine Learning**: estimar el gasto anual de un cliente a partir de su actividad en la web y evaluar qué tan buena es esa estimación.

Semana 7 · ML Supervisado: Regresión y Clasificación

Pregunta guía: ¿puedo estimar un valor o una categoría a partir de otras variables, y qué tan confiable es esa estimación?

7.1 ¿Qué es Machine Learning?

Concepto. El **aprendizaje automático** (*Machine Learning*, ML) agrupa métodos que **aprenden patrones a partir de datos** en lugar de seguir reglas escritas a mano. En lugar de programar "si visita más de 10 veces, gastará mucho", se entrega al algoritmo un historial de clientes y él estima la relación (Kelleher et al., 2020).

	Aprendizaje supervisado	Aprendizaje no supervisado
Datos	Incluyen la respuesta que se quiere predecir (variable objetivo)	No hay variable objetivo
Objetivo	Predecir un valor o una categoría	Descubrir estructura: grupos, patrones
Ejemplos de tareas	Regresión (valor numérico), clasificación (categoría)	Clustering (agrupamiento)
En RutaMarket	Estimar el gasto anual; identificar clientes de alto valor	Segmentar clientes por comportamiento (Semana 8)

7.2 El proceso CRISP-DM

Un modelo no empieza con el algoritmo. **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) es un proceso de referencia que organiza un proyecto de datos en seis fases iterativas (Chapman et al., 2000). La Unidad 2 lo recorre completo y el proyecto PMD1 lo aplica a un caso real:

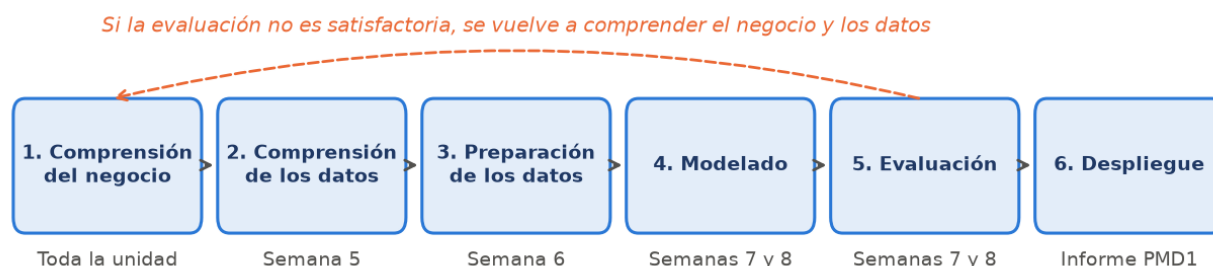


Figura 3. Fases de CRISP-DM y su correspondencia con la Unidad 2.

7.3 Datos de entrenamiento y de prueba

Concepto. Un modelo se evalúa con datos que **no vio** durante el aprendizaje. Por eso se dividen los datos en un conjunto de **entrenamiento** (para aprender) y otro de **prueba** (para evaluar), normalmente 80 % y 20 %. Evaluar con los mismos datos del entrenamiento es como calificar un examen con las preguntas que el estudiante ya conocía.

PREGUNTA DE NEGOCIO

¿Qué datos usaremos para el modelo?

CÓDIGO PYTHON

```
df = pd.read_sql_query('SELECT * FROM clientes_modelo', con)
columnas = ['visitas_web_mes', 'antigüedad_meses', 'edad', 'gasto_2025']
df[columnas].describe().round(1).loc[['mean', 'min', 'max']]

from sklearn.model_selection import train_test_split
X = df[['visitas_web_mes']]
y = df['gasto_2025']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
print(len(X_train), 'clientes para entrenar |', len(X_test), 'para evaluar')
```

RESULTADO

estadístico	visitas_web_mes	antigüedad_meses	edad	gasto_2025
mean	8.20	18.30	35.50	2,112.60
min	1.00	0.50	18.00	39.90
max	19.00	35.90	60.00	10,731.70

318 clientes para entrenar | 80 para evaluar

INTERPRETACIÓN

Se leen los datos limpios desde SQLite (Semana 6). La variable objetivo **y** es **gasto_2025**, y la variable explicativa **X**, las visitas mensuales a la web. **random_state=42** fija la división aleatoria para que el resultado sea reproducible: 318 clientes para entrenar y 80 para evaluar.

7.4 Regresión lineal simple

Concepto. La **regresión lineal** estima una variable numérica como una recta: $\hat{y} = b_0 + b_1 \cdot x$. El **intercepto** b_0 es el valor estimado cuando $x = 0$ y el **coeficiente** b_1 indica cuánto cambia $\hat{y}$, en promedio, por cada unidad adicional de x .

PREGUNTA DE NEGOCIO

¿Cuánto gasta al año un cliente según cuántas veces visita la web?

CÓDIGO PYTHON

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
reg = LinearRegression().fit(X_train, y_train)
pred = reg.predict(X_test)
mse = mean_squared_error(y_test, pred)
print('Intercepto:', round(reg.intercept_, 2))
print('Coeficiente visitas_web_mes:', round(reg.coef_[0], 2))
print('R² (prueba):', round(r2_score(y_test, pred), 3))
print('MSE:', round(mse, 1), '| RMSE:', round(mse ** 0.5, 1))
```

RESULTADO

```
Intercepto: 210.05
Coeficiente visitas_web_mes: 233.74
R² (prueba): 0.462
MSE: 2094527.4 | RMSE: 1447.2
```

INTERPRETACIÓN

Coeficiente: cada visita mensual adicional se asocia, en promedio, con **S/ 233.74 más** de gasto anual. Un cliente con 10 visitas tendría un gasto estimado de $210.05 + 233.74 \times 10 \approx \text{S/ } 2\,547$.

R² = 0.462: las visitas explican alrededor del 46 % de las diferencias de gasto entre clientes; el resto depende de factores que el modelo no incluye. **RMSE = S/ 1 447:** en promedio, la estimación se equivoca en ese monto, grande frente a un gasto medio de S/ 2 113. El modelo sirve para identificar tendencias, pero no para predecir con precisión el gasto de un cliente individual.

Métrica	Qué mide	Cómo leerla
R² (coef. de determinación)	Proporción de la variación de y explicada por el modelo	Entre 0 y 1 (en prueba puede ser negativo); más alto es mejor
MSE (error cuadrático medio)	Promedio de los errores al cuadrado	En unidades al cuadrado (soles²): útil para comparar modelos
RMSE (raíz del MSE)	Tamaño típico del error	En las mismas unidades de y (soles): fácil de comunicar

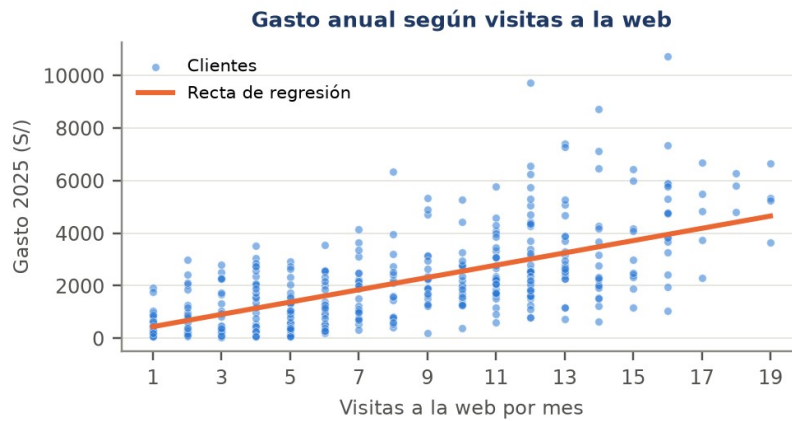


Figura 4. Gasto anual de cada cliente frente a sus visitas mensuales, con la recta estimada.

La Figura 4 muestra por qué el R^2 es moderado: la tendencia es clara, pero para un mismo número de visitas el gasto varía mucho entre clientes.

7.5 Regresión lineal múltiple

PREGUNTA DE NEGOCIO

¿Mejora la estimación si agregamos antigüedad y edad?

CÓDIGO PYTHON

```
variables = ['visitas_web_mes', 'antigüedad_meses', 'edad']
X_train, X_test, y_train, y_test = train_test_split(df[variables], y,
                                                    test_size=0.2, random_state=42)

reg2 = LinearRegression().fit(X_train, y_train)
pred2 = reg2.predict(X_test)
for v, c in zip(variables, reg2.coef_):
    print(f'{v:18s} {c:9.2f}')
print('R² (prueba):', round(r2_score(y_test, pred2), 3),
      '| RMSE:', round(mean_squared_error(y_test, pred2) ** 0.5, 1))
```

RESULTADO

```
visitas_web_mes      238.82
antigüedad_meses     -17.25
edad                 11.08
R² (prueba): 0.49 | RMSE: 1410.3
```

INTERPRETACIÓN

El R^2 apenas sube de 0.462 a 0.49 y el RMSE baja de S/ 1 447 a S/ 1 410. Las visitas siguen siendo la variable importante; la antigüedad y la edad aportan poco. Cada coeficiente se interpreta *manteniendo constantes las demás variables*: el de antigüedad (-17.25) es pequeño y no debe interpretarse como que la antigüedad reduce el gasto.

La lección para el negocio: agregar variables no garantiza un mejor modelo; conviene agregar las que tengan sentido y verificar la mejora con los datos de prueba.

7.6 Overfitting y underfitting

	Underfitting (subajuste)	Buen ajuste	Overfitting (sobreajuste)
Qué ocurre	El modelo es demasiado simple y no capta el patrón	Capta el patrón general	Memoriza el ruido de los datos de entrenamiento
Señal	Error alto en entrenamiento y en prueba	Errores similares y aceptables	Muy buen desempeño en entrenamiento y peor en prueba

PREGUNTA DE NEGOCIO

¿Qué pasa si hacemos el modelo más complejo?

CÓDIGO PYTHON

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
muestra = df.sample(40, random_state=1)
Xa, Xb, ya, yb = train_test_split(muestra[['visitas_web_mes']], muestra['gasto_2025'],
                                  test_size=0.3, random_state=0)

for grado in [1, 3, 10]:
    m = make_pipeline(PolynomialFeatures(grado), StandardScaler(), LinearRegression())
    m.fit(Xa, ya)
    r2_ent = r2_score(ya, m.predict(Xa))
    r2_pru = r2_score(yb, m.predict(Xb))
    print(f'grado {grado:2d} | R² entrenamiento: {r2_ent:.2f} | R² prueba: {r2_pru:.2f}')
```

RESULTADO

```
grado 1 | R² entrenamiento: 0.54 | R² prueba: 0.71
grado 3 | R² entrenamiento: 0.57 | R² prueba: 0.63
grado 10 | R² entrenamiento: 0.69 | R² prueba: 0.52
```

INTERPRETACIÓN

Con una muestra pequeña de 40 clientes se ajustan curvas cada vez más flexibles (polinomios de grado 1, 3 y 10). Al aumentar la complejidad, el R^2 de entrenamiento **sube** (0.54 $\rightarrow$ 0.69), pero el de prueba **baja** (0.71 $\rightarrow$ 0.52): el modelo de grado 10 se adapta a las particularidades de los 28 clientes de entrenamiento y generaliza peor. Este es el sobreajuste, y por eso la evaluación siempre se hace con datos de prueba (James et al., 2021).

7.7 Clasificación: concepto e interpretación

Concepto. Cuando la variable objetivo es una **categoría** (sí/no, alto/bajo), el problema es de **clasificación**. El modelo asigna cada caso a una clase. Aquí definimos como **cliente de alto valor** al 25 % que más gastó y usamos una **regresión logística**, un clasificador básico, con las mismas tres variables.

PREGUNTA DE NEGOCIO

¿Podemos identificar a los clientes de alto valor por su actividad?

CÓDIGO PYTHON

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
umbral = df['gasto_2025'].quantile(0.75)
df['alto_valor'] = (df['gasto_2025'] >= umbral).astype(int)
Xc_train, Xc_test, yc_train, yc_test = train_test_split(
    df[variables], df['alto_valor'], test_size=0.2,
    random_state=42, stratify=df['alto_valor'])
clf = LogisticRegression(max_iter=1000).fit(Xc_train, yc_train)
pred_c = clf.predict(Xc_test)
print('Umbral alto valor: S/', round(umbral, 2))
print('Exactitud (accuracy):', round(accuracy_score(yc_test, pred_c), 3))
print(confusion_matrix(yc_test, pred_c))
```

RESULTADO

```
Umbral alto valor: S/ 2781.92
Exactitud (accuracy): 0.825
[[58  2]
 [12  8]]
```

INTERPRETACIÓN

La **exactitud** (accuracy) es 82.5 %: 66 de 80 clientes de prueba fueron clasificados correctamente. Pero la **matriz de confusión** cuenta otra historia: de los 20 clientes que realmente son de alto valor (fila inferior), el modelo solo reconoce 8 y confunde 12 con clientes comunes. Cuando el modelo dice "alto valor", acierta en 8 de 10 casos (**precisión** del 80 %), pero solo encuentra al 40 % de los clientes de alto valor (**sensibilidad** o **recall**).

Si la empresa quiere invitar a todos sus mejores clientes a un programa exclusivo, este modelo dejaría fuera a la mayoría. Con clases desbalanceadas, una exactitud alta puede ser engañosa: siempre hay que revisar la matriz de confusión.

	Predicho: común (0)	Predicho: alto valor (1)
Real: común (0)	58 · verdaderos negativos	2 · falsos positivos
Real: alto valor (1)	12 · falsos negativos	8 · verdaderos positivos

7.8 Uso responsable de los modelos

- **Asociación no es causalidad:** que las visitas se asocien con el gasto no prueba que aumentar las visitas aumente el gasto.
- **Calidad de los datos:** un modelo entrenado con los precios mal registrados de la Semana 6 habría aprendido patrones falsos.
- **Alcance:** el modelo describe a los clientes de 2025 de RutaMarket; aplicarlo a otro periodo o mercado exige volver a evaluarlo.
- **Comunicación:** informar siempre las métricas con su significado para el negocio y las limitaciones del modelo.

Cierre de la Semana 7

Lo que debes dominar

- Distinguir aprendizaje supervisado y no supervisado, y regresión de clasificación.
- Ubicar cada paso de un proyecto de datos en las fases de CRISP-DM.
- Dividir los datos en entrenamiento y prueba y explicar por qué se evalúa con datos no vistos.
- Entrenar una regresión lineal e interpretar coeficientes, R^2 , MSE y RMSE en términos del negocio.
- Reconocer el sobreajuste y leer una matriz de confusión más allá de la exactitud.

Comprueba tu aprendizaje

1. ¿Por qué no es correcto evaluar un modelo con los mismos datos con los que se entrenó?
2. Un modelo tiene $R^2 = 0.95$ en entrenamiento y 0.30 en prueba. ¿Qué problema presenta y qué harías?
3. Explica a un gerente, sin tecnicismos, qué significa un RMSE de S/ 1 447 en el modelo de gasto.
4. Un clasificador de fraude tiene 99 % de exactitud, pero solo el 1 % de las transacciones son fraudes. ¿Por qué ese resultado podría no servir?

Conexión con la Semana 8

Semana 7 → ya sabemos predecir un valor o una categoría cuando conocemos la respuesta en los datos históricos.

Semana 8 → ¿y si no hay una respuesta que predecir? Aprenderemos a **descubrir grupos** de clientes con aprendizaje no supervisado (K-Means).

Semana 8 · ML No Supervisado: Clustering con K-Means

Pregunta guía: ¿qué grupos naturales de clientes existen y qué decisión de negocio corresponde a cada uno?

8.1 Aprendizaje no supervisado: ¿cuándo usar clustering?

Concepto. El **clustering** (agrupamiento) reúne casos parecidos entre sí y diferentes de los de otros grupos, **sin** una variable objetivo. El algoritmo no sabe qué grupos existen: los descubre a partir de las variables que se le entregan.

Contexto. Se usa cuando se necesita organizar a muchos casos para tratarlos de forma diferenciada: segmentar clientes para campañas, agrupar productos con patrones de venta similares o identificar tiendas con comportamiento parecido.

Aplicación. RutaMarket no puede diseñar una campaña distinta para cada uno de sus 398 clientes, pero sí para cuatro o cinco grupos. Usaremos tres variables de comportamiento: **cantidad de pedidos**, **ticket promedio** y **días sin comprar**, las mismas que suelen emplearse en los análisis de recencia, frecuencia y valor.

8.2 Cómo funciona K-Means

K-Means divide los datos en **k** grupos, cada uno representado por su **centroide** (el punto promedio del grupo) (MacQueen, 1967):

1. Se eligen **k** centroides iniciales al azar.
2. Cada cliente se asigna al centroide más cercano.
3. Cada centroide se recalcula como el promedio de los clientes asignados.
4. Se repiten los pasos 2 y 3 (**iteraciones**) hasta que las asignaciones dejan de cambiar: el algoritmo **converge**.

La **inercia** es la suma de las distancias al cuadrado de cada cliente a su centroide: mide qué tan compactos son los grupos. Como el resultado depende del punto de partida, `n_init=10` ejecuta el algoritmo diez veces y conserva la mejor solución.

8.3 Preparar los datos y elegir k: el método del codo

K-Means usa distancias, por lo que las variables se **estandarizan** primero (Semana 6). Para elegir **k** se calcula la inercia con distintos valores: siempre disminuye al agregar grupos, pero a partir de cierto punto la mejora es pequeña. Ese punto de quiebre es el **codo**.

PREGUNTA DE NEGOCIO

¿Cuántos segmentos conviene formar?

CÓDIGO PYTHON

```
from sklearn.cluster import KMeans
rfm = ['pedidos_2025', 'ticket_promedio', 'dias_sin_comprar']
X_esc = StandardScaler().fit_transform(df[rfm])
inercias = []
for k in range(1, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_esc)
    inercias.append(round(km.inertia_, 1))
print(inercias)
```

RESULTADO

```
[1194.0, 677.5, 456.7, 321.9, 257.6, 223.6, 194.6, 171.6]
```

INTERPRETACIÓN

La inercia cae con fuerza de **k = 1** a **k = 4** (de 1 194 a 322) y luego las reducciones son menores (258, 224...). Elegimos **k = 4**: agrega poca complejidad y los grupos siguen siendo interpretables. El codo es una guía, no una regla exacta; la decisión final también considera si los grupos tienen sentido para el negocio.

8.4 Interpretar los clusters

PREGUNTA DE NEGOCIO

¿Cómo es cada grupo de clientes?

CÓDIGO PYTHON

```
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X_esc)
df['cluster'] = km.labels_
print('Iteraciones hasta converger:', km.n_iter_)
perfil = df.groupby('cluster')[rfm + ['gasto_2025']].mean().round(1)
perfil['clientes'] = df['cluster'].value_counts().sort_index()
perfil

nombres = {0: 'Regulares', 1: 'Inactivos', 2: 'Frecuentes',
            3: 'Compras grandes ocasionales'}
df['segmento'] = df['cluster'].map(nombres)
df['segmento'].value_counts()
```

RESULTADO

Iteraciones hasta converger: 7

cluster	pedidos_2025	ticket_promedio	dias_sin_comprar	gasto_2025	clientes
0	4.70	400.90	60.50	1,904.10	114
1	2.10	207.90	268.30	440.10	98
2	12.20	254.80	29.90	3,225.30	171
3	1.80	1,029.30	212.30	1,938.90	15

segmento	count
Frecuentes	171
Regulares	114
Inactivos	98
Compras grandes ocasionales	15

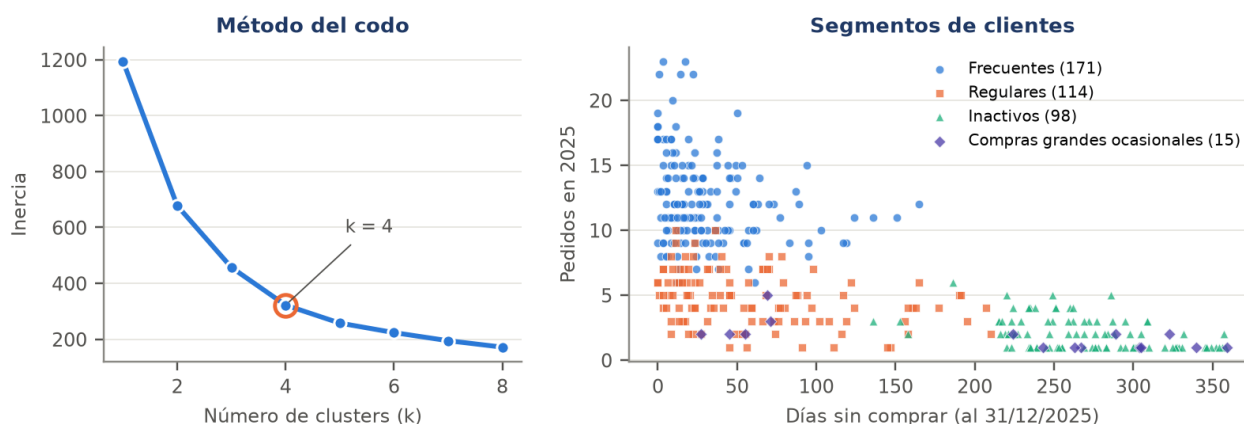


Figura 5. Método del codo (izquierda) y clientes según pedidos y días sin comprar, por segmento (derecha).

El algoritmo solo entrega números de cluster; **nombrarlos e interpretarlos** es tarea del analista, a partir de los promedios de cada grupo:

Segmento	Perfil (promedios)	Decisión de negocio sugerida
Frecuentes (171)	12.2 pedidos, ticket S/ 255, última compra hace 30 días	Programa de fidelización y beneficios por recurrencia
Regulares (114)	4.7 pedidos, ticket S/ 401, última compra hace 61 días	Recomendaciones personalizadas para aumentar la frecuencia
Inactivos (98)	2.1 pedidos, ticket S/ 208, sin comprar hace 268 días	Campaña de reactivación con incentivo acotado
Compras grandes ocasionales (15)	1.8 pedidos, ticket S/ 1 029, sin comprar hace 212 días	Atención personalizada y avisos de productos de alto valor

8.5 Guardar los segmentos en la base de datos

PREGUNTA DE NEGOCIO

¿Cuánto aporta cada segmento a las ventas?

CÓDIGO PYTHON

```
df.to_sql('clientes_modelo', con, if_exists='replace', index=False)
pd.read_sql_query('SELECT segmento, COUNT(*) AS clientes,
                  ROUND(SUM(gasto_2025), 2) AS gasto_total
                  FROM clientes_modelo
                  GROUP BY segmento
                  ORDER BY gasto_total DESC', con)
```

RESULTADO

segmento	clientes	gasto_total
Frecuentes	171	551,530.00
Regulares	114	217,069.50
Inactivos	98	43,131.20
Compras grandes ocasionales	15	29,082.90

INTERPRETACIÓN

El segmento queda guardado como una **nueva columna** de `clientes_modelo`, de modo que cualquier consulta SQL posterior puede usarlo, como la verificación mostrada. Los 171 clientes frecuentes generan S/ 551 530, el 65.6 % del gasto total; los 98 inactivos solo el 5.1 %.

Para la gerencia, el mensaje es claro: proteger a los clientes frecuentes es prioritario y reactivar a los inactivos tiene potencial, pero partiendo de un gasto bajo. Así, el resultado del análisis vuelve a la base de datos y queda disponible para otras áreas.

8.6 Limitaciones de K-Means

- Hay que elegir k de antemano; el método del codo ayuda, pero no siempre muestra un quiebre claro.
- Es sensible a la escala (por eso se estandariza) y a los valores atípicos, que pueden desplazar los centroides.
- Tiende a formar grupos compactos y de forma aproximadamente esférica; grupos alargados o de tamaños muy distintos pueden quedar mal separados.
- Los segmentos describen comportamientos pasados: deben actualizarse cuando cambian los datos.

Cierre de la Semana 8

Lo que debes dominar

- Explicar cuándo usar clustering y en qué se diferencia de un modelo supervisado.
- Describir el funcionamiento de K-Means: centroides, asignación, iteraciones y convergencia.
- Estandarizar variables antes de aplicar K-Means y justificar la elección de k con el método del codo.
- Interpretar los perfiles de cada cluster, nombrarlos y proponer decisiones de negocio.
- Guardar los segmentos en la base de datos y reconocer las limitaciones del método.

Comprueba tu aprendizaje

1. ¿Por qué K-Means necesita que las variables estén en la misma escala? ¿Qué ocurriría con el ticket promedio si no se estandariza?
2. En el método del codo, ¿por qué no se elige simplemente el k con menor inercia?
3. Un cluster tiene ticket alto pero pocos pedidos y mucho tiempo sin comprar. ¿Qué acción comercial propondrías y cómo medirías su resultado?
4. ¿Qué ventaja tiene guardar el segmento como columna en la base de datos en lugar de dejarlo solo en el notebook?

Hacia la evaluación de la Unidad 2

Semana 8 es también la semana de evaluación de la unidad (Pa2) y de entrega del proyecto PMD1, que integra todo lo aprendido. La siguiente sección resume ese recorrido completo.

Integración de la Unidad 2

De la base de datos a una decisión respaldada por datos limpios y modelos interpretables.

Las cuatro semanas forman un solo flujo de trabajo, que continúa el de la Unidad 1. Cada etapa depende de la anterior: un modelo entrenado con datos sucios produce conclusiones falsas, y un modelo sin interpretación no ayuda a decidir.

Mapa del recorrido

Etapas	Herramienta	Semana	Pregunta que responde
Extraer	SQL + <code>pd.read_sql_query()</code>	U1 · S5	¿Qué datos necesito y cómo los combino?
→ Explorar (EDA)	<code>info()</code> , <code>describe()</code> , <code>value_counts()</code> , gráficos	S5	¿Cómo son los datos y qué problemas tienen?
→ Limpiar	<code>to_numeric()</code> , <code>drop_duplicates()</code> , <code>fillna()</code> , IQR	S6	¿Cómo corrijo los problemas sin distorsionar la realidad?
→ Transformar	<code>groupby()</code> , <code>StandardScaler()</code>	S6	¿Qué tabla y qué escala necesita el modelo?
→ Predecir	<code>LinearRegression()</code> , <code>LogisticRegression()</code>	S7	¿Puedo estimar un valor o una categoría?
→ Segmentar	<code>KMeans()</code>	S8	¿Qué grupos existen?
→ Guardar y decidir	<code>to_sql()</code> + SQL	S6 · S8	¿Cómo pongo el resultado al servicio de la organización?

Caso integrador

La gerencia de RutaMarket dispone de un presupuesto limitado para una **campaña de reactivación** y pregunta: *¿a quiénes debemos dirigirla y qué podemos esperar?* Responderla requiere todo el recorrido de la unidad:

- 1. Explorar (Semana 5).** El EDA muestra que la base tiene clientes con muy distinta actividad y detecta problemas: duplicados, edades ocultas como texto y montos sospechosos. Sin corregirlos, contaríamos clientes dos veces y sobrestimaríamos el gasto.
- 2. Limpiar (Semana 6).** Tras eliminar duplicados y corregir los precios mal digitados, el gasto de 2025 es confiable y se construye la tabla `clientes_modelo`, con días sin comprar, pedidos y ticket promedio.
- 3. Segmentar (Semana 8).** K-Means identifica 98 clientes **inactivos** (sin comprar hace unos 268 días en promedio) y 15 clientes de **compras grandes ocasionales** (ticket de S/ 1 029). Son los candidatos naturales a la campaña, con mensajes distintos para cada grupo.
- 4. Estimar (Semana 7).** El modelo de regresión indica que el gasto se asocia con las visitas a la web. Por eso, un indicador razonable del éxito de la campaña es el aumento de las visitas de los clientes reactivados, sin prometer un monto exacto: el RMSE recuerda que las estimaciones individuales son imprecisas.
- 5. Decidir y comunicar.** La recomendación se sustenta en datos: priorizar a los inactivos con un incentivo acotado y ofrecer atención personalizada a los compradores de alto ticket, midiendo el resultado con los mismos indicadores en el siguiente periodo. El segmento queda guardado en la base de datos para que el área de marketing lo consulte.

Relación con el proyecto PMD1

Fase del proyecto	Semana	Qué aplica de esta unidad
Fase 1: problema de negocio + EDA	6 (checkpoint)	Carga del dataset en SQLite, lectura con pandas, EDA e interpretación de gráficos (Semana 5)
Fase 2: limpieza y modelado	7	Pipeline de limpieza documentado (Semana 6) y modelo supervisado evaluado con datos de prueba (Semana 7)
Fase 3: entrega final	8	Comparación de modelos, segmentación (Semana 8), informe gerencial con recomendaciones y código reproducible

Checklist de dominio de la Unidad 2

Marca cada afirmación solo si puedes demostrarla en un notebook propio:

- ☐ Leo datos desde SQLite con una consulta SQL y desde un archivo CSV, y los cargo en un DataFrame.
- ☐ Exploro un DataFrame con `info()`, `describe()` y `value_counts()` y redacto los hallazgos en celdas de texto.
- ☐ Construyo e interpreto gráficos de distribución, correlación y valores nulos.
- ☐ Detecto nulos ocultos, tipos incorrectos, textos inconsistentes y duplicados, y los corrijo justificando cada decisión.
- ☐ Elijo una estrategia de imputación (media, mediana, moda o categoría explícita) según el tipo y la distribución de la variable.
- ☐ Detecto atípicos con boxplot, IQR y z-score, y verifico si son errores antes de modificarlos.
- ☐ Normalizo variables y guardo los datos limpios como nuevas tablas en SQLite.
- ☐ Entreno una regresión lineal con datos de entrenamiento y prueba e interpreto coeficientes, R^2 , MSE y RMSE.
- ☐ Reconozco el sobreajuste y leo una matriz de confusión más allá de la exactitud.
- ☐ Aplico K-Means, elijo k con el método del codo, interpreto los segmentos y los guardo en la base de datos.

Conclusiones

1. Python y pandas no reemplazan a SQL, lo complementan: SQL extrae y combina los datos en la base y pandas los explora, transforma y prepara para el análisis. El flujo termina devolviendo los resultados a la base de datos.
2. El análisis exploratorio es el primer control de calidad: permitió descubrir en RutaMarket nulos ocultos como texto, ciudades escritas de varias formas, duplicados y montos imposibles antes de que distorsionaran cualquier conclusión.
3. Limpiar datos exige criterio: un valor atípico puede ser un error o un buen cliente, y cada imputación es un supuesto. Las decisiones deben justificarse con el negocio y documentarse.
4. Un modelo supervisado se evalúa con datos que no vio y se interpreta en el lenguaje del negocio. Métricas como R^2 , RMSE o la matriz de confusión muestran no solo cuánto acierta el modelo, sino también en qué se equivoca.
5. El clustering convierte datos de comportamiento en segmentos accionables, pero sus grupos solo tienen valor cuando el analista los interpreta, les da nombre y los vincula a decisiones concretas.
6. La calidad de un modelo depende de la calidad de los datos y de su gestión: sin la base construida en la Unidad 1, ninguno de estos análisis sería confiable.

Referencias bibliográficas

- Bruce, P., Bruce, A., y Gedeck, P. (2020). *Practical statistics for data scientists* (2.^a ed.). O'Reilly Media.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., y Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.
- Gironés Roig, J., Casas Roma, J., Minguillón Alfonso, J., y Caihuelas Quiles, R. (2017). *Minería de datos: modelos y algoritmos*. Editorial UOC.
- Grus, J. (2019). *Data science from scratch: First principles with Python* (2.^a ed.). O'Reilly Media.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Ilyas, I. F., y Chu, X. (2019). *Data cleaning*. ACM Books.
- James, G., Witten, D., Hastie, T., y Tibshirani, R. (2021). *An introduction to statistical learning* (2.^a ed.). Springer.
- Kelleher, J. D., Mac Namee, B., y D'Arcy, A. (2020). *Fundamentals of machine learning for predictive data analytics* (2.^a ed.). MIT Press.

- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. En *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281–297). University of California Press.
- McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Provost, F., y Fawcett, T. (2013). *Data science for business*. O'Reilly Media.
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>